

Ruta a .NET senior

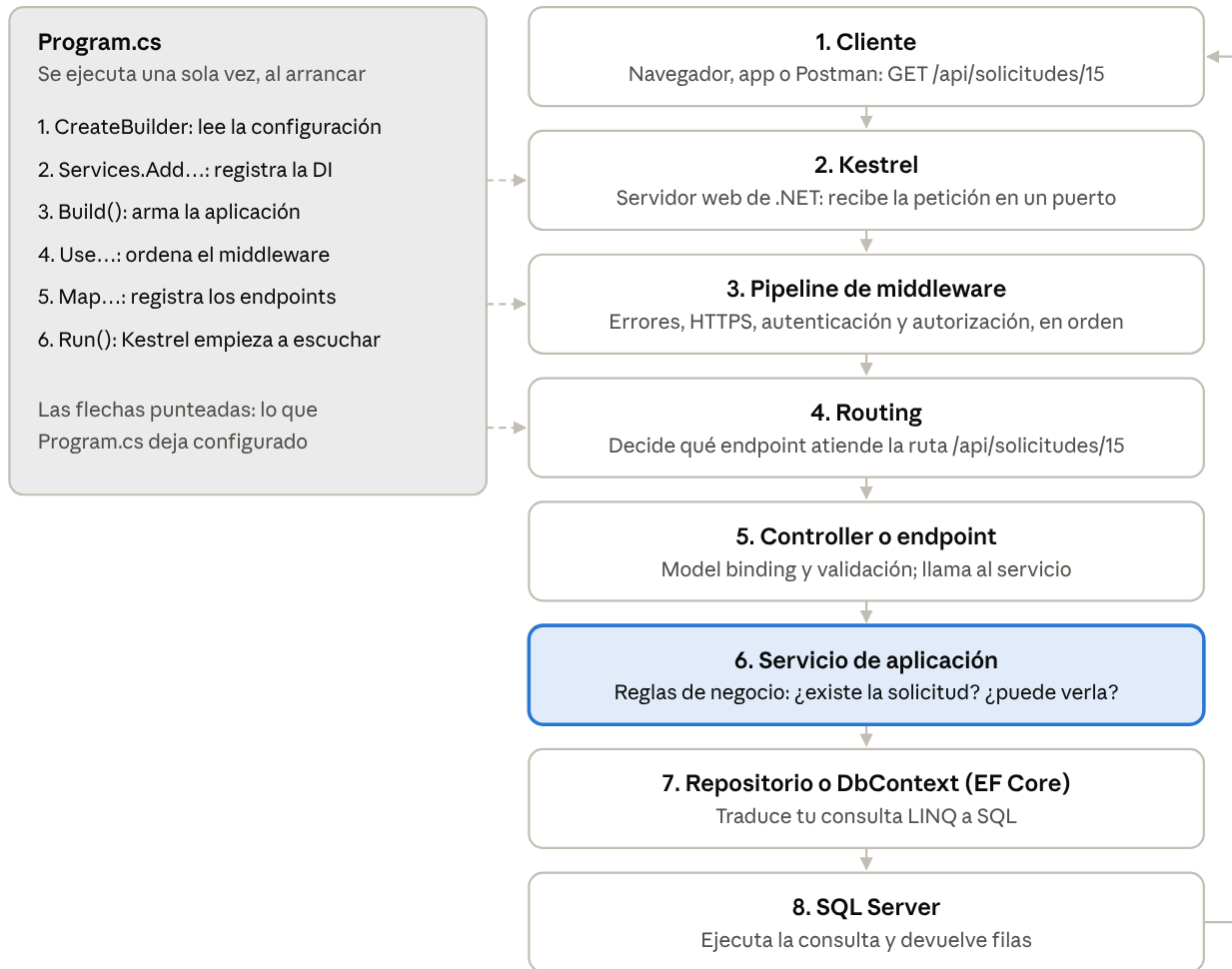
Sep 30, 2026 · @Alex

Después del plan de 12 semanas te faltan 12 temas para ser un .NET senior, y el primero es entender de punta a punta cómo fluye y se estructura una aplicación completa. Los demás van ordenados por cuánto pesan en el trabajo diario y en las entrevistas senior; recorrerlos toma unos 12 meses al mismo ritmo de ~12 horas por semana.

El flujo de una aplicación completa

Una aplicación ASP.NET Core vive en dos momentos. **Al arrancar**, `Program.cs` registra los servicios y arma el pipeline una sola vez. **En cada petición**, el mensaje baja por 8 capas hasta la base de datos y la respuesta sube de regreso.

Cada petición cruza ocho capas; tu lógica de negocio vive en una sola



Línea de la derecha: la respuesta (JSON + código HTTP) sube por las mismas capas en orden inverso

Ciclo de vida de una petición en ASP.NET Core · arranque y 8 capas

La regla de oro de la estructura: **cada capa solo conoce a la de abajo, y la lógica de negocio no depende de la web ni de la base de datos.** Así puedes cambiar SQL Server o probar una regla sin levantar el servidor.

Cómo se ve eso en carpetas

Proyecto	Qué contiene	Ejemplo en tu sistema de crédito	Depende de
<code>src/Credito.Api</code>	<code>Program.cs</code> , controllers o endpoints, middleware, configuración	<code>SolicitudesController</code> , <code>appsettings.json</code>	Application
<code>src/Credito.Application</code>	Casos de uso, servicios, DTOs, interfaces (puertos)	<code>AprobarSolicitudService</code> , <code>ISolicitudRepository</code>	Domain
<code>src/Credito.Domain</code>	Entidades y reglas de negocio puras, sin EF ni HTTP	<code>LineaDeCredito.Disponer(monto)</code> valida el saldo disponible	nada
<code>src/Credito.Infrastructure</code>	EF Core, repositorios, clientes de APIs externas, correo	<code>CreditoDbContext</code> , <code>KobraClient</code> , <code>TwilioSender</code>	Application
<code>tests/</code>	Pruebas unitarias e integración	<code>LineaDeCreditoTests</code> , <code>SolicitudesApiTests</code>	lo que prueban

Esta forma se conoce como **Clean Architecture**: las flechas de dependencia apuntan hacia el dominio. En tu código de DiSí probablemente la lógica vive dentro de los controllers de MVC; reconocer esa diferencia y saber explicarla ya es una conversación de nivel senior.

Cómo aprenderlo: en la semana 5 del carril de repos sigues un flujo en `dotnet/eShop`. Después del plan, sobre tu repo `SolicitudesApi`, pon un *breakpoint* en cada capa de tu propia API y recorre una petición con el depurador de Visual Studio, capa por capa, anotando qué objeto existe en cada paso.

Los 12 temas, por peso

El orden refleja qué tanto pesa cada tema en lo que hace un senior todos los días: diseñar, mantener datos sanos, asegurar la calidad y resolver problemas en producción. Los primeros cuatro son el núcleo; sin ellos, lo demás no se sostiene.

#	Tema	Por qué pesa	Lo dominas cuando puedes...	Tiempo
1	Flujo y arquitectura de una aplicación completa	Todo senior diseña o ajusta la estructura de lo que construye el equipo	Explicar cada capa de tu app, moverla a Clean Architecture y justificar qué va en cada proyecto	6 semanas
2	Diseño de código: SOLID, patrones y refactorización	La diferencia entre código que funciona y código que se puede mantener 5 años	Tomar un método de 300 líneas y refactorizarlo con pruebas sin romper nada	4 semanas
3	Datos a fondo: SQL Server y EF Core avanzado	La mayoría de los problemas de producción son de datos: consultas lentas, bloqueos, migraciones	Leer un plan de ejecución, manejar concurrencia optimista y aplicar una migración sin tumbar el sistema	5 semanas
4	Pruebas y calidad	Un senior responde por la calidad del equipo, no solo de su código	Escribir pruebas unitarias y de integración con Testcontainers, y hacer code review útil	4 semanas
5	Rendimiento y concurrencia en .NET	Separar al que "hace que funcione" del que sabe por qué está lento	Perfilar una API, explicar el GC, usar caché bien y medir con BenchmarkDotNet	4 semanas
6	Seguridad de aplicaciones	En fintech un error de seguridad cuesta dinero y licencias	Revisar una API contra el OWASP Top 10 e integrar Microsoft Entra ID	3 semanas
7	Observabilidad y operación en producción	A los seniors los llaman cuando algo falla a las 2 de la mañana	Diagnosticar un error con logs, métricas y trazas de OpenTelemetry sin tocar el servidor	3 semanas

#	Tema	Por qué pesa	Lo dominas cuando puedes...	Tiempo
8	Cloud y DevOps en Azure (AZ-204)	La mayoría de las vacantes senior piden nube y despliegue automatizado	Desplegar tu app con CI/CD, Key Vault, Service Bus e infraestructura como código (Bicep)	6 semanas
9	Sistemas distribuidos y mensajería	Las apps grandes se integran por eventos, no solo por llamadas HTTP	Implementar el patrón Outbox, consumidores idempotentes y reintentos	4 semanas
10	System design	Es la entrevista que define si te contratan como senior	Diseñar en 45 minutos un sistema como "originación de crédito para 100 mil solicitudes al mes"	4 semanas
11	Habilidades senior: comunicación, mentoría e inglés técnico	Un senior multiplica al equipo; el inglés abre vacantes mejor pagadas	Escribir un ADR, estimar un proyecto, guiar a un junior y sostener una entrevista técnica en inglés	continuo
12	IA aplicada en .NET	Diferenciador en 2026: pocos devs .NET integran IA en productos	Construir una función con <code>Microsoft.Extensions.AI</code> o Semantic Kernel que resuelva un caso real	3 semanas

Lo que quedó fuera a propósito: microservicios con Kubernetes (mucha infraestructura para aprender poco diseño) y otro framework de frontend (ya eres full stack; como senior .NET pesa más el backend).

En qué orden recorrerlos durante 2027

Todo se practica sobre **un solo proyecto de portafolio** que crece cada trimestre: un sistema de crédito revolvente como monolito modular. Así terminas el año con un repo que demuestra los 12 temas, en lugar de 12 ejercicios sueltos.

Trimestre	Temas	Cómo crece el proyecto
Ene-mar	1. Flujo y arquitectura · 2. Diseño de código	Estructura en capas (Api, Application, Domain, Infrastructure), casos de uso y primeros ADRs
Abr-jun	3. Datos a fondo · 4. Pruebas	SQL optimizado, concurrencia en disposiciones de línea, pruebas con Testcontainers
Jul-sep	5. Rendimiento · 6. Seguridad · 7. Observabilidad	Caché, benchmarks, revisión OWASP, login con Entra ID, OpenTelemetry
Oct-dic	8. Azure · 9. Mensajería · 10. System design	Despliegue con Bicep y CI/CD, eventos con Service Bus y Outbox; certificación AZ-204
Todo el año	11. Habilidades senior · 12. IA aplicada	Inglés técnico semanal, un ADR por decisión importante; módulo de IA al final

Si consigues trabajo nuevo a inicios de 2027, conserva el orden pero ajusta el ritmo: los primeros meses en un empleo nuevo también son aprendizaje, y los temas 1 a 4 los vas a practicar ahí mismo.

Cómo saber que ya eres senior

Los años de experiencia no definen al senior; lo definen el alcance y la autonomía. Revisa esta lista cada trimestre:

- ☐ Tomas un requerimiento ambiguo y lo conviertes en un diseño técnico que otros pueden implementar
- ☐ Explicas las ventajas y desventajas de cada decisión, no solo la opción que elegiste
- ☐ Encuentras la causa de un error en producción usando logs, métricas y trazas
- ☐ Tus code reviews mejoran el código de otros sin frenar al equipo
- ☐ Detectas riesgos de seguridad, rendimiento y datos antes de que lleguen a producción
- ☐ Un junior avanza más rápido por trabajar contigo
- ☐ Pasas una entrevista de system design y otra técnica en inglés
- ☐ Tu repo de portafolio demuestra los 12 temas y lo puedes defender línea por línea